

Niech będzie dany wycinek bazy danych D pochodzący z przychodni lekarskiej, która zawiera cztery atrybuty oznaczające etykietę czasu ($Data$) oraz odpowiednio identyfikator pacjenta ($pacjentID$), recepty ($receptaID$), i przypisanego lekarstwa.

Przyjmując próg odcięcia ($minsup$) na poziomie 0,25, należy znaleźć częste wzorce sekwencji oraz maksymalne wzorce sekwencji, stosując algorytm *AprioriAll*.

Tab.1. Baza danych D

Data	PacjentID	ReceptaId	Lek_id
01/26/98	105	112	30
01/26/98	106	113	10
01/26/98	106	113	20
02/02/98	105	114	90
02/10/98	106	115	30
03/08/98	106	116	40
03/08/98	106	116	60
03/08/98	106	116	70
03/09/98	200	117	30
03/09/98	200	117	50
03/09/98	200	117	70
03/09/98	220	118	30
03/10/98	300	119	90
03/15/98	220	120	40
03/15/98	220	120	70
03/18/98	220	121	90

Rozwiązanie

Krok 1. Sortowanie i grupowanie.

Pierwszym atrybutem po którym należy posortować zbiór baza danych D jest $PacjentID$, drugim $Data$. Wynik operacji sortowania przedstawia Tabela 1. Następnie należy ją pogrupować, biorąc pod uwagę atrybut $PacjentID$. Wynik tej operacji przedstawia Tabela 2.

Tab.2. Baza danych sekwencji D_s

pacjentID	lek_id
105	<(30) (90)>
106	<(10, 20) (30) (40, 60, 70)>
200	<(30, 50, 70)>
220	<(30) (40, 70) (90)>
300	<(90)>

Należy zauważyć, iż baza D_S zawiera pięć sekwencji w obrębie których można wyróżnić sześć zbiorów 1-elementowych, dwa zbiory 2-elementowe oraz 2 zbiory 3-elementowe, gdzie przyjmuje się iż elementy zostały posortowane w porządku rosnącym.

Krok2. Znajdowanie częstych zbiorów.

Do znalezienia częstych zbiorów możemy zastosować dowolny algorytm odkrywania częstych zbiorów. W analizowanym przypadku zastosujemy algorytm *Apriori*.

Zbiór $C_1 = \{(10), (20), (30), (40), (50), (60), (70), (90)\}$. Obliczamy wsparcie dla każdego zbioru i otrzymujemy następujące wyniki: (30): 0,80; (40): 0,40; (70): 0,60; (90): 0,60; tym samym $L_1 = \{(30), (40), (70), (90)\}$, czyli $\#L_1 = 4$. Następnie generujemy zbiór $C_2 = \{(30, 40), (30, 70), (30, 90), (40, 70), (40, 90), (70, 90)\}$, czyli $\#C_2 = 6$. Obliczamy wsparcie dla każdego zbioru i otrzymujemy następujące wyniki: (30, 40): 0; (30, 70): 0,20; (30, 90): 0; (40, 70): 0,40; (40, 90): 0; (70, 90): 0; tym samym $L_2 = \{(40, 70)\}$, czyli $\#L_2 = 1$. Następnie generujemy zbiór C_3 ; pierwszym i zarazem jedynym możliwym do wygenerowania zbiorem 3-elementowym jest (40, 70, 90); jednak jego podzbiór 2-elementowy (70, 90) nie jest częsty; tym samym $C_3 = \emptyset$, co oznacza iż algorytm zakończył działanie.

Wynikiem działania algorytmu *Apriori* jest rodzina zbiorów częstych, sformowana ze zbiorów L_1 oraz L_2 .

Krok 3. Transformacja.

W pierwszym kroku dokonujemy transformacji bazy danych sekwencji D_S do uporządkowanej listy wyrazów D_{TS} , zgodnie z tabelą 3.

Tab. 3. Zbiory częste wraz z odwzorowaniem w zbiór liczb naturalnych

zbiór częsty	wsparcie	odwzorowanie
(30)	0,80	1
(40)	0,40	2
(70)	0,60	3
(40, 70)	0,40	4
(90)	0,60	5

Celem transformacji jest zastąpienie każdego wyrazu T sekwencji S uporządkowaną listą zbiorów częstych zawierających się w wyrazie T . W konsekwencji usuwane są z wyrazu T wszystkie te elementy, które nie są częste (nie mogą wystąpić we wzorcach sekwencji). Jeżeli sekwencja nie zawiera żadnych zbiorów częstych to nie wchodzi ona do przetransformowanej bazy danych, jednak brana jest pod uwagę przy obliczaniu wsparcia wzorca sekwencji.

Tab. 4. Baza danych po transformacji D_{TS}

sekwencja początkowa	po transformacji	po odwzorowaniu
<(30) (90)>	<((30)) ((90))>	<(1) (5)>
<(10, 20) (30) (40, 60, 70)>	<((30)) ((40), (70), (40 70))>	<(1) (2 3 4)>
<(30, 50, 70)>	<((30), (70))>	<(1, 3)>
<(30) (40, 70) (90)>	<((30)) ((40), (70), (40 70)) ((90))>	<(1) (2 3 4) (5)>
<(90)>	<((90))>	<(5)>

Weźmy pod uwagę sekwencję drugą w postaci $\langle (10, 20), (30), (40, 60, 70) \rangle$; na początku zawiera ona jeden zbiór 2-elementowy $(10, 20)$, następnie jeden zbiór 1-elementowy (30) oraz jeden zbiór 3-elementowy $(40, 60, 70)$; usuwamy z niej zbiory nieczęste tj. (10) , (20) i (60) ; tym samym przyjmie ona postać $\langle (30), (40, 70) \rangle$, czyli posiada ona jeden zbiór 1-elementowy oraz jeden 2-elementowy $(40, 70)$; następnie dokonujemy transformacji sekwencji w taki sposób, że każdy wyraz zastępujemy uporządkowaną listą zbiorów częstych; tym samym zbiór $(40, 70)$ podzielimy na dwa zbiory 1-elementowe oraz jeden 2-elementowy; ostatecznie sekwencja po transformacji przyjmie postać $\langle ((30)) ((40), (70), (40\ 70)) \rangle$, zaś po odwzorowaniu $\langle (1) (2\ 3\ 4) \rangle$.

Krok 4. Sekwencjonowanie.

Zbiór sekwencji częstych oznaczamy przez LS_1 jest wynikiem kroku znajdowania zbiorów częstych; w analizowanym przypadku $L_1 = \{(30), (40), (70), (90)\}$, zaś $L_2 = \{(40, 70)\}$; tym samym po odwzorowaniu (patrz tab. 3) zbiór $LS_1 = \{1, 2, 3, 4, 5\}$.

Do znalezienia kolejnych częstych sekwencji posłuży funkcja *apriori_generate()*. Na podstawie zbiorów częstych generowane są sekwencje częste (wzorce sekwencji); generowanie sekwencji częstych realizowane jest zgodnie z algorytmem *AprioriAll*. Przez LS_k oznaczmy sekwencje częste o długości k , przez CS_k sekwencje kandydujące o długości k ; k -sekwencje kandydujące generowane są na podstawie $(k-1)$ sekwencji częstych w dwóch etapach funkcji *apriori_generate()*:

- **etap połączenia:** operacja połączenia dwóch sekwencji ze zbioru LS_{k-1} w celu wygenerowania k -sekwencji kandydującej; warunkiem połączenia dwóch sekwencji LS_{k-1} jest zgodność pierwszych $(k-2)$ wyrazów obu sekwencji;
- **etap eliminacji:** usuwane są wszystkie sekwencje $S \in CS_k$ takie dla których istnieją $(k-1)$ -podsekwencje sekwencji S nienależące do LS_{k-1} .

Z LS_1 został wygenerowany zbiór 2-sekwencji kandydujących CS_2 dla których zostało policzone wsparcie (tabela 5).

Tab. 5. Zestawienie 2- sekwencji kandydujących

sekwencja	wsparcie
$\langle 1\ 1 \rangle$	0
$\langle 1\ 2 \rangle$	0,40
$\langle 1\ 3 \rangle$	0,40
$\langle 1\ 4 \rangle$	0,40
$\langle 1\ 5 \rangle$	0,40
$\langle 2\ 1 \rangle$	0
$\langle 2\ 2 \rangle$	0
$\langle 2\ 3 \rangle$	0
$\langle 2\ 4 \rangle$	0
$\langle 2\ 5 \rangle$	0,2
$\langle 3\ 1 \rangle$	0
$\langle 3\ 2 \rangle$	0
$\langle 3\ 3 \rangle$	0
$\langle 3\ 4 \rangle$	0
$\langle 3\ 5 \rangle$	0,2
$\langle 4\ 1 \rangle$	0
$\langle 4\ 2 \rangle$	0
$\langle 4\ 3 \rangle$	0
$\langle 4\ 4 \rangle$	0
$\langle 4\ 5 \rangle$	0,2
$\langle 5\ 1 \rangle$	0
$\langle 5\ 2 \rangle$	0
$\langle 5\ 3 \rangle$	0
$\langle 5\ 4 \rangle$	0

<5 5>	0
-------	---

Tym samym moc zbioru 2-sekwencji kandydujących jest równy 25. Biorąc pod uwagę próg odcięcia tylko cztery sekwencje posiadają wymagane wsparcie, tj.: <1 2>, <1 3>, <1 4> i <1 5>, czyli $\#LS_2 = 4$.

W następnym kroku Funkcja *apriori_generate()* generuje na etapie połączenia zbiór sekwencji kandydujących:

<1 2 2>, <1 2 3>, <1 2 4>, <1 2 5>,
 <1 3 2>, <1 3 3>, <1 3 4>, <1 3 5>,
 <1 4 2>, <1 4 3>, <1 4 4>, <1 4 5>,
 <1 5 2>, <1 5 3>, <1 5 4>, <1 5 5>.

Zbiór 3-sekwencji kandydujących CS_3 jest zbiorem pustym, ponieważ każda z ww. zawiera 2-sekwencje nienależące do zbioru LS_2 . Tym samym algorytm kończy działanie.

Krok 5. Maksymalizacja (opcjonalny).

W tym kroku usuwamy z LS_2 te sekwencje, które nie są maksymalne; sekwencje <1 2> oraz <1 3> nie są maksymalne, gdyż zawierają się w sekwencji <1 4>, ponieważ zbiory (30) i (40) to odpowiednio pierwszy wyraz i pierwszy element drugiego wyrazu czwartej sekwencji, zaś zbiór (70) to drugi element drugiego wyrazu tej sekwencji. Ostatecznie zbiór maksymalnych wzorców sekwencji dla $minsup \geq 0,25$ składa się z dwóch wzorców <(30) (90)>: 0,4 oraz <(30) (40, 70)>: 0,4.